

Array

BY BAHARUDIN OSMAN

Baharudin Osman
Baharudin Osman

Contents

- • Introduction to array
- • Array declaration
- • Array initialization
- • Assigning values
- • Reading values

Introduction to array $\Rightarrow$ store multiple value of single variables

- Arrays are used to store multiple values in a single variable, instead of declaring separate variables for each value.
- To create an array, define the data type (like int) and specify the name of the array followed by square brackets [].
- To insert values to it, use a comma-separated list inside curly braces, and make sure all values are of the same data type:

```
int myNumbers[] = {25, 50, 75, 100};
```

Avoid Mixing Data Types

- It is important to note that all elements in an array must be of the same data type.
- This means you cannot mix different types of values, like integers and floating point numbers, in the same array:

```
int myArray[] = {25, 50, 75, 3.15, 5.99};
```

- In the example above, the values 3.15 and 5.99 will be truncated to 3 and 5. In some cases it might also result in an error, so it is important to always make sure that the elements in the array are of the same type.

✓ Array Initialization

int age[] = {20, 30}

- Two ways to initialize an array.

```
int myNumbers[] = {25, 50, 75, 100};
```

```
// Declare an array of four integers:
```

```
int myNumbers[4];
```

```
// Add elements
```

```
myNumbers[0] = 25;
```

```
myNumbers[1] = 50;
```

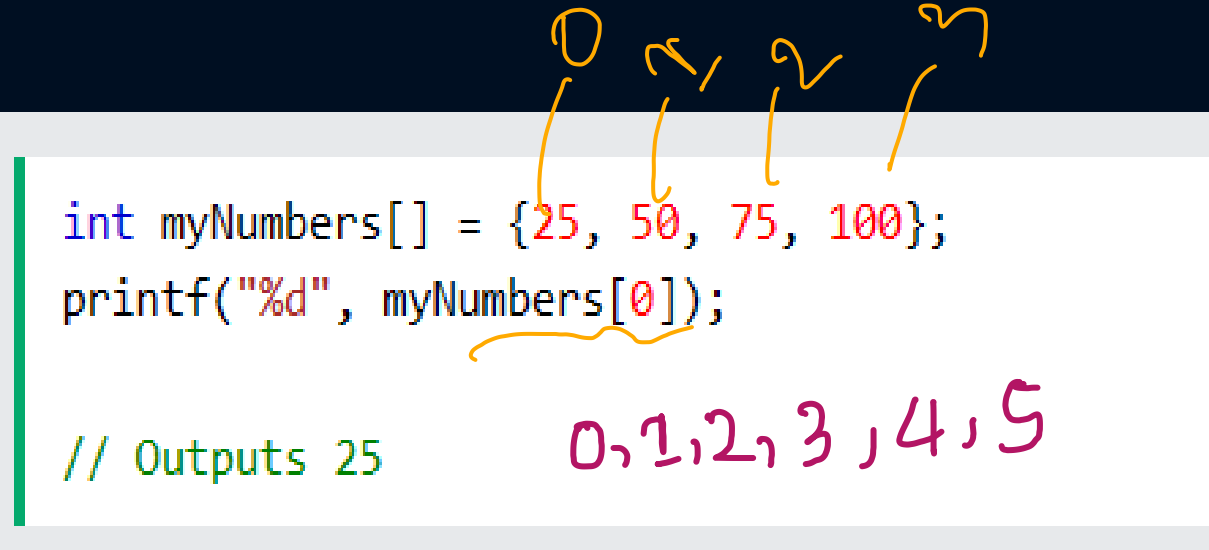
```
myNumbers[2] = 75;
```

```
myNumbers[3] = 100;
```

- Using this method, **you should know the number of array elements in advance**, in order for the program to store enough memory.
- You are not able to change the size of the array after creation.

Access the Elements of an Array

- To access an array element, refer to its index number.
- Array indexes start with 0: [0] is the first element. [1] is the second element, etc.
- This statement accesses the value of the first element [0] in myNumbers:



```
int myNumbers[] = {25, 50, 75, 100};  
printf("%d", myNumbers[0]);
```

```
// Outputs 25
```

0, 1, 2, 3, 4, 5

Change an Array Element

myNumbers[3] = 89;

- To change the value of a specific element, refer to the index number:

```
myNumbers[0] = 33;
```

```
int myNumbers[] = {25, 50, 75, 100};
```

```
myNumbers[0] = 33;
```

```
printf("%d", myNumbers[0]);
```

```
// Now outputs 33 instead of 25
```

✓ Loop Through an Array

- ✓ You can loop through the array elements with the for loop.
- The following example outputs all elements in the myNumbers array:

```
int myNumbers[] = {25, 50, 75, 100};  
int i;  
  
for (i = 0; i < 4; i++) {  
    printf("%d\n", myNumbers[i]);  
}
```

0 1 2 3
25 50 75 100
Reserve an
Array

Get Array Size or Length

- To get the size of an array, you can use the **sizeof** operator:

```
int myNumbers[] = {10, 25, 50, 75, 100};  
printf("%lu", sizeof(myNumbers)); // Prints 20
```

- Why did the result show 20 instead of 5, when the array contains 5 elements?
- It is because the sizeof operator returns the size of a type in bytes.

- to find out how many elements an array has, you can use the following formula (which divides the size of the array by the size of the first element in the array):

```
int myNumbers[] = {10, 25, 50, 75, 100};  
int length = sizeof(myNumbers) / sizeof(myNumbers[0]);  
  
printf("%d", length); // Prints 5
```

$\text{length} = \frac{\text{sizeof}(\text{Numbers})}{\text{sizeof}(\text{Numbers}[0])}$

$20/4 = 5$ element

Making Better Loops

- Previously, we wrote the size of the array in the loop condition ($i < 4$).
- This is not ideal, since it will only work for arrays of a specified size.
- Its better to write the code as below.

```
int myNumbers[] = {25, 50, 75, 100};  
int length = sizeof(myNumbers) / sizeof(myNumbers[0]);  
int i;  
  
for (i = 0; i < length; i++) {  
    printf("%d\n", myNumbers[i]);  
}
```

you can also
put length there

Example 1 : Find an average

```
#include <stdio.h>

int main() {

    // An array storing different ages

    int ages[] = {20, 22, 18, 35, 48, 26, 87, 70};

    float avg, sum = 0;

    int i;

    // Get the length of the array

    int length = sizeof(ages) / sizeof(ages[0]);

    // Loop through the elements of the array and
    accumulate the sum
```

```
    for (i = 0; i < length; i++) {
        sum += ages[i];
    }

    // Calculate the average by dividing the sum by the
    length
    avg = sum / length;

    // Print the average
    printf("The average age is: %.2f", avg);

    return 0;
}
```

Example 2 : Find the lowest value

find the lowest value of an array

```
// An array storing different ages

int ages[] = {20, 22, 18, 35, 48, 26, 87, 70};

int i;

// Get the length of the array

int length = sizeof(ages) / sizeof(ages[0]);

// Create a variable and assign the first
array element of ages to it

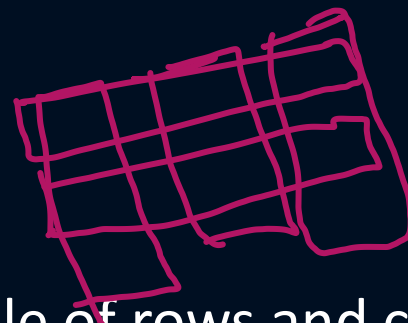
int lowestAge = ages[0];

// Loop through the elements of the ages array
to find the lowest age
```

```
for (i = 0; i < length; i++) {
    if (lowestAge > ages[i]) {
        lowestAge = ages[i];
    }
}
```

I need the length

Two-Dimensional Arrays



Handwritten notes in pink:

	C0	C1	C2
R0	1	4	2
R1	3	6	8

- A 2D array is also known as a matrix (a table of rows and columns).
- To create a 2D array of integers, take a look at the following example:

```
int matrix[2][3] = { {1, 4, 2}, {3, 6, 8} };
```

	COLUMN 0	COLUMN 1	COLUMN 2
ROW 0	1	4	2
ROW 1	3	6	8

	0	1	2
0	1	4	2
1	3	6	8

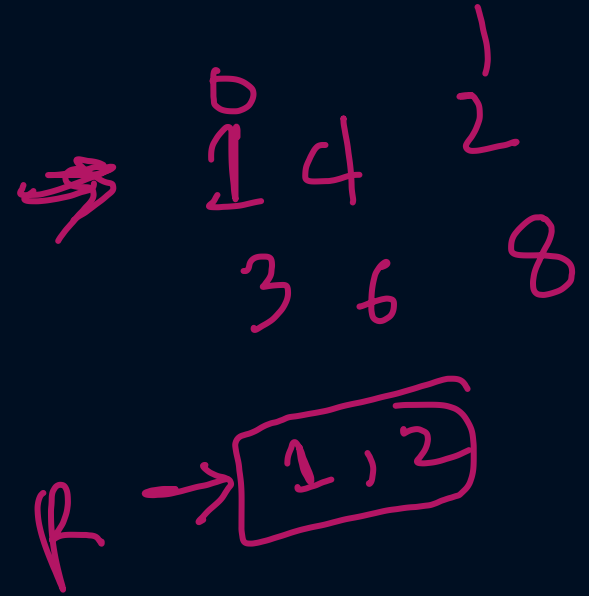
✓ Access the Elements of a 2D Array

- To access an element of a two-dimensional array, you must specify the index number of both the row and column.
- This statement accesses the value of the element in the first row (0) and third column (2) of the matrix array.

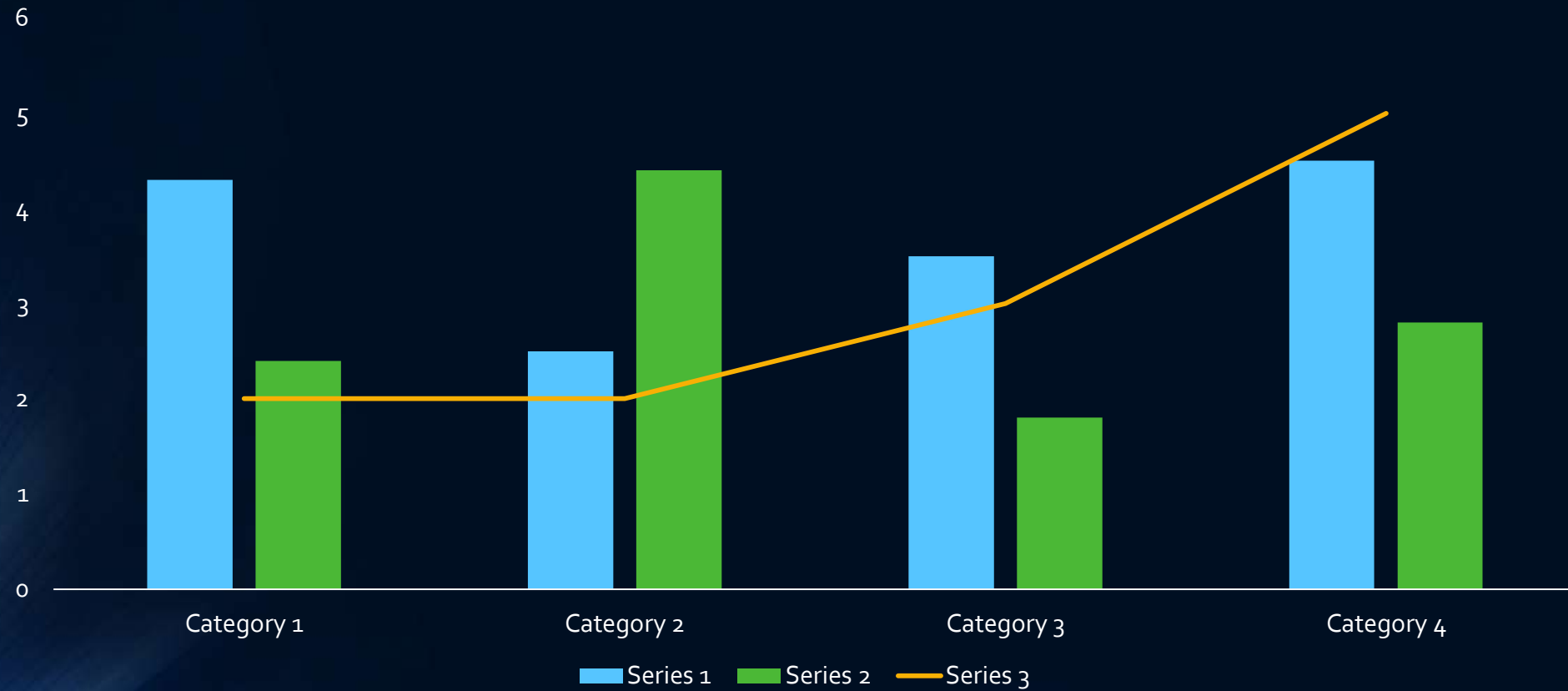
```
int matrix[2][3] = { {1, 4, 2}, {3, 6, 8} };  
printf("%d", matrix[0][2]); // Outputs 2
```

- Remember that: Array indexes start with 0:
[0] is the first element
[1] is the second element, etc.

matrix



Title and Content Layout with Chart



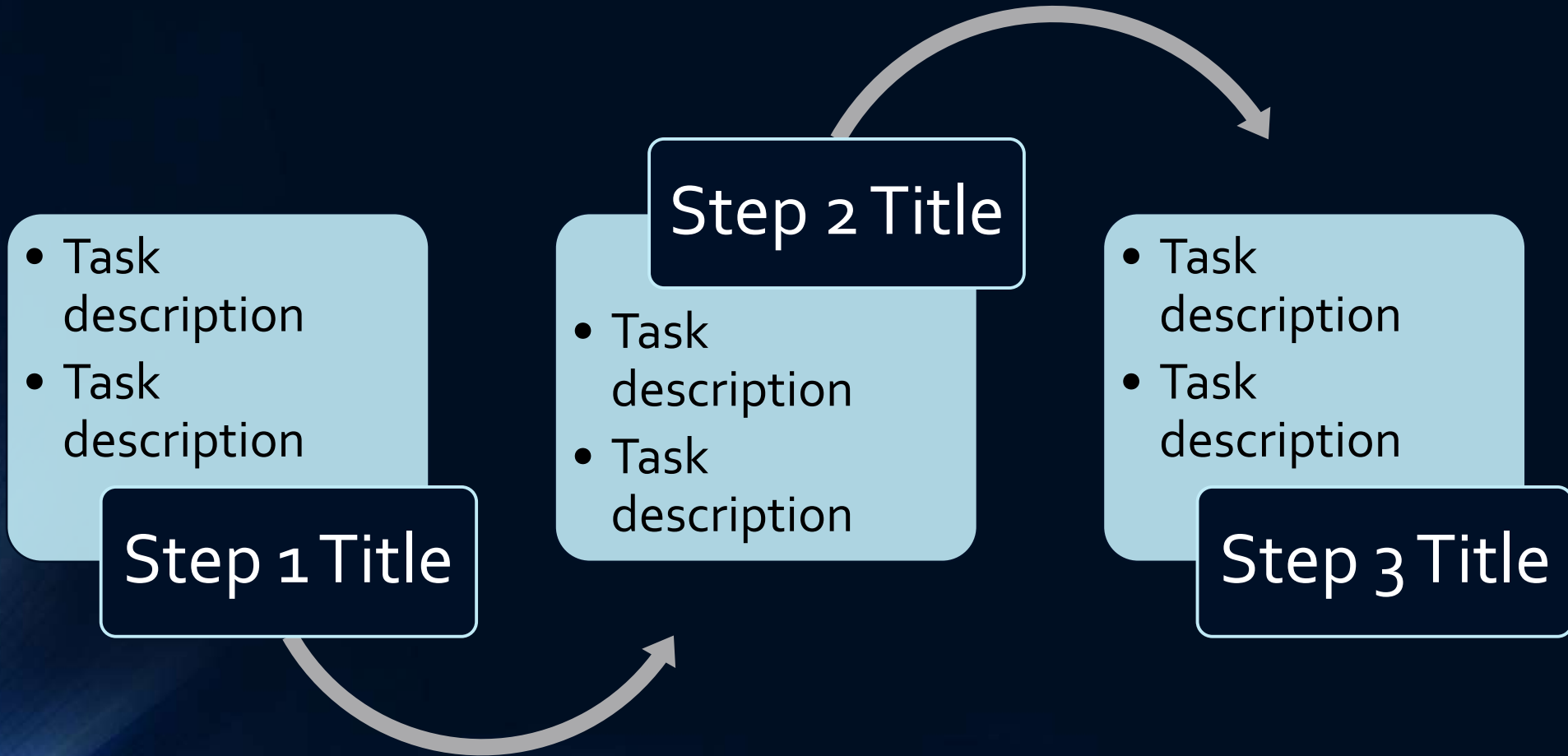


Two Content Layout with Table

- 
- First bullet point here
 - Second bullet point here
 - Third bullet point here

Class	Group 1	Group 2
Class 1	82	95
Class 2	76	88
Class 3	84	90

Title and Content Layout with SmartArt



Add a Slide Title - 1

Add a Slide Title - 2

Add a Slide Title - 3

Add a Slide Title

- 4

Add a Slide Title

- 5

